Namma-Santhe Ledger App

Code:

1. Data Models (Kotlin Data Classes)

All application entities are represented as Kotlin data classes. These map directly to the Room database and Firestore documents.

1.1 Core Entities

```
// Customer.kt
package com.nammasanthe.ledger.data.entity

import androidx.room.Entity
import androidx.room.PrimaryKey

@Entity(tableName = "customers")
data class Customer(
    @PrimaryKey(autoGenerate = true) val id: Long = 0,
    val name: String,
    val phone: String,
    val address: String = "",
    val photoPath: String? = null,
    val createdAt: Long = System.currentTimeMillis()
)
```

```
// Transaction.kt not found
```

```
// ScanConfirmation.kt not found
```

2. Database Layer (Room & DAO)

Room Database implementation for offline-first capabilities, handling all local CRUD operations.

2.1 Room Database & DAOs

```
// AppDatabase.kt not found
```

```
// CustomerDao.kt
package com.nammasanthe.ledger.data.dao

import androidx.room.*
import com.nammasanthe.ledger.data.entity.Customer
import kotlinx.coroutines.flow.Flow

@Dao
interface CustomerDao {
    @Query("SELECT * FROM customers ORDER BY name COLLATE NOCASE ASC")
    fun observeAll(): Flow<List<Customer>>

    @Query("SELECT * FROM customers WHERE id = :id")
    suspend fun getById(id: Long): Customer?

    @Query("SELECT * FROM customers WHERE name LIKE '%' || :q || '%' OR phone LIKE
'%' || :q || '%'")
    fun search(q: String): Flow<List<Customer>>

    @Insert
    suspend fun insert(customer: Customer): Long

    @Update
    suspend fun update(customer: Customer)

    @Delete
    suspend fun delete(customer: Customer)
}
```

```
// TransactionDao.kt
package com.nammasanthe.ledger.data.dao

import androidx.room.*
import com.nammasanthe.ledger.data.entity.TxnEntity
import kotlinx.coroutines.flow.Flow

data class CustomerBalance(
    val customerId: Long,
    val name: String,
    val phone: String,
    val photoPath: String?,
    val totalCredit: Double,
    val totalPayment: Double,
    val balance: Double,
    val lastTxn: Long?
)

@Dao
interface TransactionDao {
    @Query("SELECT * FROM transactions WHERE customerId = :customerId ORDER BY
date DESC")
    fun observeByCustomer(customerId: Long): Flow<List<TxnEntity>>

    @Query("SELECT * FROM transactions ORDER BY date DESC LIMIT :limit")
    fun observeRecent(limit: Int = 20): Flow<List<TxnEntity>>

    @Query("SELECT * FROM transactions WHERE date >= :since ORDER BY date ASC")
    fun observeSince(since: Long): Flow<List<TxnEntity>>

    @Query("SELECT * FROM transactions WHERE syncPending = 1")
    suspend fun pendingSync(): List<TxnEntity>

    @Query("""
        SELECT IFNULL(SUM(CASE WHEN type = 'CREDIT' THEN amount ELSE -amount END),
0)
        FROM transactions
    """)
    fun observeOutstanding(): Flow<Double>

    @Query("""
        SELECT c.id AS customerId, c.name AS name, c.phone AS phone, c.photoPath
AS photoPath,
            IFNULL(SUM(CASE WHEN t.type = 'CREDIT' THEN t.amount ELSE 0 END), 0)
AS totalCredit,
            IFNULL(SUM(CASE WHEN t.type = 'PAYMENT' THEN t.amount ELSE 0 END), 0)
AS totalPayment,
            IFNULL(SUM(CASE WHEN t.type = 'CREDIT' THEN t.amount ELSE -t.amount
END), 0) AS balance,
            MAX(t.date) AS lastTxn
        FROM customers c
        LEFT JOIN transactions t ON t.customerId = c.id
        GROUP BY c.id
        ORDER BY balance DESC
    """)
    fun observeBalances(): Flow<List<CustomerBalance>>

    @Query("SELECT * FROM transactions WHERE id = :id LIMIT 1")
    suspend fun getById(id: Long): TxnEntity?

    @Query("SELECT * FROM transactions ORDER BY date DESC")
    suspend fun getAll(): List<TxnEntity>

    @Insert
    suspend fun insert(txn: TxnEntity): Long

    @Update
    suspend fun update(txn: TxnEntity)

    @Delete
    suspend fun delete(txn: TxnEntity)
}
```

3. OCR Pipeline & Document Scanning

Dual-engine OCR implementation using Gemini ML and Tesseract for Kannada text parsing.

3.1 OCR Orchestrator & Parsers

```
// OcrPipeline.kt
package com.nammasanthe.ledger.ocr

import android.content.Context
import android.graphics.Bitmap
import com.nammasanthe.ledger.translation.TranslationManager
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.withContext
import java.text.SimpleDateFormat
import java.util.Locale

/**
 * OCR Pipeline — orchestrates ML Kit and Tesseract (SimpleKannadaOcr) for
 * optimal text extraction from bill/ledger images.
 *
 * Strategy:
 * - For Kannada: Run both ML Kit (numbers/prices) + Tesseract (Kannada text),
 *   then merge results for best coverage.
 * - For Hindi/English: ML Kit is primary, Tesseract is fallback.
 */
class OcrPipeline(context: Context) {

    private val mlKit = MlKitOcr()
    private val simpleOcr = SimpleKannadaOcr(context)
    private var preferredLanguage: OcrLanguage = OcrLanguage.UNKNOWN

    fun setPreferredLanguage(language: OcrLanguage) {
        preferredLanguage = language
    }

    suspend fun run(bitmap: Bitmap): OcrResult = withContext(Dispatchers.Default)
{
        // Step 1: Run ML Kit (fast, good for numbers & Latin text)
        val (mlText, mlConf) = runCatching { mlKit.recognize(bitmap,
preferredLanguage) }
            .getOrDefault("" to 0f)

        var bestText = mlText
        var bestConf = mlConf
        var source = "ml-kit"
        var warning: String? = null
        val detected = detectLanguage(mlText)

        // Step 2: For Kannada or when ML Kit is insufficient, run Tesseract
        val needsSimpleOcr = preferredLanguage == OcrLanguage.KANNADA ||
            detected == OcrLanguage.KANNADA ||
            mlText.isBlank() ||
            mlConf < 0.5f

        if (needsSimpleOcr) {
            val sResult = runCatching { simpleOcr.recognize(bitmap, "kan+eng") }
            val (sText, sConf) = sResult.getOrDefault("" to 0f)
            if (sResult.isFailure) {
                val msg = sResult.exceptionOrNull()?.message?.takeIf {
it.isNotBlank() }
                warning = msg ?: "Simple OCR failed."
            }

            if (sText.isNotBlank()) {
                if (preferredLanguage == OcrLanguage.KANNADA || detected ==
OcrLanguage.KANNADA) {
                    // For Kannada: Merge ML Kit numbers with Tesseract Kannada
text
                    bestText = mergeResults(mlText, sText)
                    bestConf = maxOf(mlConf, sConf)
                    source = "merged-ocr"
                } else if (sText.length > bestText.length || sConf > bestConf) {
                    bestText = sText
                    bestConf = sConf
                    source = "simple-ocr"
                }
            }
        }

        val finalLang = if (preferredLanguage != OcrLanguage.UNKNOWN)
preferredLanguage
                        else detectLanguage(bestText)
        val amount = extractAmount(bestText)
        val date = parseDate(bestText)
```

```
        OcrResult(bestText.trim(), finalLang, bestConf, source, amount, date,
warning)
    }

    /**
     * Merge ML Kit result (good for numbers/English) with Tesseract result
     * (good for Kannada script). Keeps the Tesseract text as base but
     * supplements with any prices/amounts ML Kit found that Tesseract missed.
     */
    private fun mergeResults(mlKitText: String, tesseractText: String): String {
        if (mlKitText.isBlank()) return tesseractText
        if (tesseractText.isBlank()) return mlKitText

        // If Tesseract result already has good numbers, just use it
        val tessAmounts = AMOUNT_REGEX.findAll(tesseractText).count()
        val mlAmounts = AMOUNT_REGEX.findAll(mlKitText).count()

        if (tessAmounts >= mlAmounts) return tesseractText

        // Tesseract has Kannada text but is missing prices — append ML Kit
amounts
        val mlAmountLines = mlKitText.lines().filter { line ->
            AMOUNT_REGEX.containsMatchIn(line)
        }

        return if (mlAmountLines.isNotEmpty()) {
            tesseractText + "\n" + mlAmountLines.joinToString("\n")
        } else {
            tesseractText
        }
    }

    fun close() {
        mlKit.close()
        simpleOcr.close()
    }

    companion object {
        // Enhanced amount regex: handles Rs., ₹, INR, plain numbers, and x/-
format
        private val AMOUNT_REGEX = Regex(
            """(?i)(?:rs\.?\s*|₹\s*|inr\s*)?(\d{1,3}(?:[,\d]{0,9})
(?:\.\d{1,2})?)\s*(?:/[--])?"""
        )

        fun detectLanguage(text: String): OcrLanguage {
            if (text.isBlank()) return OcrLanguage.UNKNOWN
            val kannada = text.count { it.code in 0x0C80..0x0CFF }
            val devanagari = text.count { it.code in 0x0900..0x097F }
            val latin = text.count { it.isLetter() && it.code < 0x80 }
            return when {
                kannada > 0 && kannada >= devanagari && kannada >= latin ->
OcrLanguage.KANNADA
                devanagari > 0 && devanagari >= latin -> OcrLanguage.HINDI
                latin > 0 -> OcrLanguage.ENGLISH
                else -> OcrLanguage.UNKNOWN
            }
        }

        /**
         * Extract the most likely total amount from OCR text.
         * Priority: explicit total line > largest number > last number.
         */
        private fun extractAmount(text: String): Double? {
            // First check for explicit "total" lines
            val totalLine = text.lines().firstOrNull { line ->
                line.contains("total", ignoreCase = true) ||
                line.contains("ಒಟ್ಟು", ignoreCase = true) || // Kannada "total"
                line.contains("ಮೊತ್ತ", ignoreCase = true)    // Kannada "amount"
            }

            if (totalLine != null) {
                val amount = AMOUNT_REGEX.find(totalLine)
                    ?.groupValues?.get(1)
                    ?.replace(",", "")
                    ?.toDoubleOrNull()
                if (amount != null && amount > 0) return amount
            }

            // Fallback: find the largest reasonable amount
```

```
            val allAmounts = AMOUNT_REGEX.findAll(text)
                .mapNotNull { it.groupValues[1].replace(",", "").toDoubleOrNull()
}
                .filter { it > 0 && it < 100000 } // Reasonable range for bills
                .toList()

            return allAmounts.maxOrNull()
        }

        private val DATE_FORMATS = listOf("dd/MM/yyyy", "dd-MM-yyyy", "yyyy-MM-
dd", "dd MMM yyyy")

        private fun parseDate(text: String): Long? {
            val match = Regex("""\b(\d{1,2}[\-/ ]\d{1,2}[\-/
]\d{2,4})\b""").find(text) ?: return null
            for (fmt in DATE_FORMATS) {
                try {
                    val parsed = SimpleDateFormat(fmt,
Locale.ENGLISH).parse(match.value) ?: continue
                    return parsed.time
                } catch (_: Throwable) {}
            }
            return null
        }
    }

    suspend fun translateIfNeeded(
        result: OcrResult,
        translator: TranslationManager
    ): String? {
        if (result.rawText.isBlank()) return null
        // ML Kit on-device translation supports Hindi -> English. Kannada is not
in ML Kit's
        // language set, so for Kannada we leave the original text and let the
user edit.
        val src = when (result.language) {
            OcrLanguage.HINDI -> "hi"
            else -> return null
        }
        return runCatching { translator.translateIfDownloaded(result.rawText, src,
"en") }.getOrNull()
    }
}
```

```
// BillParser.kt
package com.nammasanthe.ledger.ocr

import com.nammasanthe.ledger.translation.KannadaOfflineTranslator

/**
 * Multi-line bill parser — the public entry point for OCR post-processing.
 *
 * Typical flow:
 *  1. [OcrPipeline] produces raw text + detected language.
 *  2. [OcrViewModel] translates (Hindi → ML Kit, Kannada →
[KannadaOfflineTranslator]).
 *  3. Caller passes translated text to [parseTranslated] (or raw text + language
to [parse]).
 *  4. Returns a ranked list of [BillItem]s ready for display / transaction
creation.
 *
 * Thread-safe (no mutable state).
 */
class BillParser(private val kannadaTranslator: KannadaOfflineTranslator) {

    companion object {
        /** Items with confidence below this are discarded by default. */
        const val MIN_CONFIDENCE = 0.20f
    }

    // ── Primary entry points ──────────────────────────────────────────────

    /**
     * Parse [rawText] in any supported language.
     * Applies offline Kannada translation when [language] is KANNADA.
     * For Hindi / English the text is expected to already be translated upstream.
     */
    fun parse(rawText: String, language: OcrLanguage): List<BillItem> {
        if (rawText.isBlank()) return emptyList()
        val working = when (language) {
            OcrLanguage.KANNADA -> kannadaTranslator.translate(rawText)
            else                -> rawText
        }
        return parseTranslated(working)
    }

    /**
     * Parse text that has already been translated to English (or is mixed
English).
     * This is the preferred entry point from [OcrViewModel].
     */
    fun parseTranslated(translatedText: String): List<BillItem> {
        if (translatedText.isBlank()) return emptyList()

        val items = translatedText
            .lines()
            .asSequence()
            .map    { it.trim() }
            .filter { it.isNotBlank() }
            .mapNotNull { BillLineParser.parse(it) }
            .filter { it.confidence >= MIN_CONFIDENCE || it.amount > 0 }
            .toList()

        // If no items were parsed, try more aggressive parsing
        if (items.isEmpty()) {
            return parseAggressively(translatedText)
        }

        return items
    }

    /**
     * More aggressive parsing for difficult OCR text.
     * Attempts to extract items even with poor OCR quality.
     */
    private fun parseAggressively(text: String): List<BillItem> {
        val items = mutableListOf<BillItem>()

        // Split by common separators and try to parse each part
        val parts = text.split(Regex("""[,\n;]"""))

        for (part in parts) {
            val cleanPart = part.trim()
            if (cleanPart.isBlank()) continue
```

```
            // Try to extract numbers and words
            val numbers = Regex("""\d+(?:\.\d+)?""").findAll(cleanPart).map {
it.value }.toList()
            val words = cleanPart.split(Regex("""\s+""")).filter { it.isNotBlank()
&& !it.matches(Regex("""\d+(?:\.\d+)?""")) }

            if (numbers.isNotEmpty() && words.isNotEmpty()) {
                // Last number is likely the amount
                val amount = numbers.lastOrNull()?.toDoubleOrNull()?.toInt() ?: 0
                val itemName = words.take(3).joinToString(" ").trim() // Take
first 3 words as item name

                if (itemName.isNotBlank() && amount > 0) {
                    items.add(BillItem(
                        item = itemName.replaceFirstChar { it.uppercase() },
                        originalLine = cleanPart,
                        quantity = "unknown",
                        amount = amount,
                        confidence = 0.3f // Lower confidence for aggressive
parsing
                    ))
                }
            }
        }

        return items
    }

    // ── Aggregation helpers ──────────────────────────────────────────────

    /** Sum of [BillItem.amount] across all items. */
    fun grandTotal(items: List<BillItem>): Int = items.sumOf { it.amount }

    /**
     * One-line summary string suitable for a transaction note.
     *
     * Example: "Onion 1kg ₹50, Banana 1kg ₹40, Potato ₹30 | Total ₹120"
     */
    fun summarize(items: List<BillItem>, maxItems: Int = 5): String {
        if (items.isEmpty()) return ""
        val parts = items.take(maxItems).map { i ->
            buildString {
                append(i.item)
                if (i.quantity != "unknown") append(" ${i.quantity}")
                if (i.amount > 0)            append(" ₹${i.amount}")
            }
        }
        val total = grandTotal(items)
        val suffix = if (items.size > maxItems) " +${items.size - maxItems} more"
else ""
        return parts.joinToString(", ") + suffix +
               if (total > 0) " | Total ₹$total" else ""
    }

    /**
     * Produce a JSON-style string (no external dependency) for logging / export.
     *
     * Output matches the spec format:
     * [{"item":"Onion","quantity":"1kg","amount":50}, ...]
     */
    fun toJsonString(items: List<BillItem>): String {
        if (items.isEmpty()) return "[]"
        val rows = items.joinToString(",\n  ") { i ->
            """
{"item":"${i.item.replace("\"","'")}","quantity":"${i.quantity}","amount":${i.amount}}"""
        }
        return "[\n  $rows\n]"
    }
}
```

4. Security & Synchronization

QR-based verification system and Firebase integration for cloud sync.

4.1 QR Generator & Sync Manager

```
// QrGenerator.kt
package com.nammasanthe.ledger.security

import android.graphics.Bitmap
import android.graphics.Color
import com.google.zxing.BarcodeFormat
import com.google.zxing.EncodeHintType
import com.google.zxing.qrcode.QRCodeWriter
import com.google.zxing.qrcode.decoder.ErrorCorrectionLevel
import com.nammasanthe.ledger.data.entity.TxnEntity
import java.net.URLEncoder
import java.util.UUID

/**
 * Generates tamper-evident QR payloads and renders them as Bitmaps.
 *
 * QR now encodes a Firebase Hosting URL so that any phone's camera
 * can open the confirmation page — no Namma-Santhe app needed on
 * the customer's device.
 *
 * Error correction: H (30 %) — tolerates partial obscuring.
 */
object QrGenerator {

    const val QR_TTL_MS = 60_000L

    /** Base URL for the Firebase-hosted confirmation page. */
    private const val CONFIRM_BASE_URL = "https://namma-santhe-
ledger.web.app/confirm"

    /**
     * Build a signed [QrPayload] for [txn].
     *
     * @param prevHash Pass the previous transaction's hash to chain records;
     *                 leave empty for standalone confirmation.
     */
    fun buildPayload(txn: TxnEntity, prevHash: String = ""): QrPayload {
        val now   = System.currentTimeMillis()
        val nonce = UUID.randomUUID().toString()
        val hash  = HashUtil.txnHash(
            txnId     = txn.id.toString(),
            amount    = txn.amount,
            type      = txn.type.name,
            timestamp = now,
            prevHash  = prevHash
        )
        return QrPayload(
            txnId     = txn.id.toString(),
            amount    = txn.amount,
            type      = txn.type.name,
            timestamp = now,
            nonce     = nonce,
            expiresAt = now + QR_TTL_MS,
            hash      = hash
        )
    }

    /**
     * Build a Firebase Hosting URL for the confirmation web page.
     *
     * @param payload The QR payload with transaction details
     * @param userId  The Firebase Auth UID of the vendor — used by the web page
     *                to write the confirmation into the correct Firestore path.
     * @param vendorName Optional vendor business name shown to customer.
     */
    fun buildConfirmUrl(
        payload    : QrPayload,
        userId     : String,
        vendorName : String = "Namma Santhe Vendor"
    ): String {
        val encodedVendor = URLEncoder.encode(vendorName, "UTF-8")
        return "$CONFIRM_BASE_URL" +
                "?txnId=${payload.txnId}" +
                "&amt=${payload.amount}" +
                "&type=${payload.type}" +
                "&hash=${payload.hash}" +
                "&ts=${payload.timestamp}" +
                "&uid=$userId" +
                "&vendor=$encodedVendor"
    }
```

```
    /**
     * Render a URL as a QR code Bitmap of [sizePx] × [sizePx].
     * Must be called on a background thread.
     */
    fun generateBitmapFromUrl(url: String, sizePx: Int = 512): Bitmap {
        val hints = mapOf(
            EncodeHintType.ERROR_CORRECTION to ErrorCorrectionLevel.H,
            EncodeHintType.MARGIN           to 1,
            EncodeHintType.CHARACTER_SET    to "UTF-8"
        )
        val matrix = QRCodeWriter().encode(url, BarcodeFormat.QR_CODE, sizePx,
sizePx, hints)
        val bmp    = Bitmap.createBitmap(sizePx, sizePx, Bitmap.Config.ARGB_8888)
        for (x in 0 until sizePx) {
            for (y in 0 until sizePx) {
                bmp.setPixel(x, y, if (matrix[x, y]) Color.BLACK else Color.WHITE)
            }
        }
        return bmp
    }

    // ── Legacy helpers (kept for backward compat with existing scan screen) ──

    /**
     * Generate a user-friendly QR code text that contains both readable info and
JSON data.
     * Format: "Namma Santhe Payment | Amount: ₹XX.XX | Type: CREDIT/PAYMENT |
[JSON]"
     */
    fun generateUserFriendlyQrText(payload: QrPayload): String {
        val amountStr = "₹${"%.2f".format(payload.amount)}"
        val typeStr = if (payload.type == "CREDIT") "Credit" else "Payment"
        val jsonPart = payload.toJson()
        return "Namma Santhe $typeStr | Amount: $amountStr | ID: ${payload.txnId}
| [$jsonPart]"
    }

    /**
     * Render [payload] as a square Bitmap of [sizePx] × [sizePx].
     * Uses user-friendly format instead of raw JSON.
     * Must be called on a background thread.
     */
    fun generateBitmap(payload: QrPayload, sizePx: Int = 512): Bitmap {
        val qrText = generateUserFriendlyQrText(payload)
        val hints = mapOf(
            EncodeHintType.ERROR_CORRECTION to ErrorCorrectionLevel.H,
            EncodeHintType.MARGIN           to 1
        )
        val matrix = QRCodeWriter().encode(qrText, BarcodeFormat.QR_CODE, sizePx,
sizePx, hints)
        val bmp    = Bitmap.createBitmap(sizePx, sizePx, Bitmap.Config.ARGB_8888)
        for (x in 0 until sizePx) {
            for (y in 0 until sizePx) {
                bmp.setPixel(x, y, if (matrix[x, y]) Color.BLACK else Color.WHITE)
            }
        }
        return bmp
    }

    /**
     * Extract JSON data from user-friendly QR text.
     */
    fun extractJsonFromUserFriendlyQr(qrText: String): String? {
        val startIndex = qrText.indexOf('[')
        val endIndex = qrText.lastIndexOf(']')
        return if (startIndex != -1 && endIndex != -1 && endIndex > startIndex) {
            qrText.substring(startIndex + 1, endIndex)
        } else {
            if (qrText.trim().startsWith("{") && qrText.trim().endsWith("}")) {
                qrText
            } else {
                null
            }
        }
    }
}
```

```
// FirebaseSyncManager.kt
package com.nammasanthe.ledger.sync

import android.content.Context
import android.net.ConnectivityManager
import android.net.NetworkCapabilities
import com.google.firebase.firestore.FirebaseFirestore
import com.google.firebase.firestore.SetOptions
import com.google.firebase.firestore.ktx.firestore
import com.google.firebase.ktx.Firebase
import com.nammasanthe.ledger.data.entity.Customer
import com.nammasanthe.ledger.data.entity.TxnEntity
import com.nammasanthe.ledger.data.repo.AppProfile
import com.nammasanthe.ledger.util.PhotoProofManager
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.tasks.await
import kotlinx.coroutines.withContext
import java.text.SimpleDateFormat
import java.util.Date
import java.util.Locale

/**
 * Manages cloud sync with Firebase Firestore.
 * Works offline-first - queues changes and syncs when online.
 */
class FirebaseSyncManager private constructor(context: Context) {

    private val db: FirebaseFirestore = Firebase.firestore
    private val appContext = context.applicationContext
    private val authManager = FirebaseAuthManager.getInstance(context)

    private val _syncState = MutableStateFlow<SyncState>(SyncState.Idle)
    val syncState: StateFlow<SyncState> = _syncState.asStateFlow()

    private val _lastSyncTime = MutableStateFlow<Long?>(null)
    val lastSyncTime: StateFlow<Long?> = _lastSyncTime.asStateFlow()

    companion object {
        @Volatile
        private var instance: FirebaseSyncManager? = null

        fun getInstance(context: Context): FirebaseSyncManager {
            return instance ?: synchronized(this) {
                instance ?: FirebaseSyncManager(context.applicationContext).also {
                    instance = it
                }
            }
        }
    }

    init {
        // Enable offline persistence
        db.firestoreSettings = com.google.firebase.firestore.firestoreSettings {
            isPersistenceEnabled = true
        }
    }

    /**
     * Check if device is online.
     */
    fun isOnline(): Boolean {
        val connectivityManager =
appContext.getSystemService(Context.CONNECTIVITY_SERVICE)
            as ConnectivityManager
        val network = connectivityManager.activeNetwork ?: return false
        val capabilities = connectivityManager.getNetworkCapabilities(network) ?:
return false
        return
capabilities.hasCapability(NetworkCapabilities.NET_CAPABILITY_INTERNET)
    }

    /**
     * Sync all data to cloud.
     */
    suspend fun syncAllData(
        profile: AppProfile,
        customers: List<Customer>,
```

```
        transactions: List<TxnEntity>
    ): Result<Unit> = withContext(Dispatchers.IO) {
        if (!authManager.isSignedIn()) {
            return@withContext Result.failure(Exception("User not signed in"))
        }

        if (!isOnline()) {
            return@withContext Result.failure(Exception("No internet connection"))
        }

        _syncState.value = SyncState.Syncing

        try {
            val userId = authManager.getCurrentUserId()!!

            // Sync profile
            syncProfile(userId, profile)

            // Sync customers
            syncCustomers(userId, customers)

            // Sync transactions
            syncTransactions(userId, transactions)

            _lastSyncTime.value = System.currentTimeMillis()
            _syncState.value = SyncState.Success

            Result.success(Unit)
        } catch (e: Exception) {
            _syncState.value = SyncState.Error(e.message ?: "Sync failed")
            Result.failure(e)
        }
    }

    /**
     * Download data from cloud.
     */
    suspend fun downloadFromCloud(): Result<CloudData> =
withContext(Dispatchers.IO) {
        if (!authManager.isSignedIn()) {
            return@withContext Result.failure(Exception("User not signed in"))
        }

        if (!isOnline()) {
            return@withContext Result.failure(Exception("No internet connection"))
        }

        _syncState.value = SyncState.Syncing

        try {
            val userId = authManager.getCurrentUserId()!!

            // Get profile
            val profileDoc = db.collection("users")
                .document(userId)
                .collection("profile")
                .document("main")
                .get()
                .await()

            val profile = if (profileDoc.exists()) {
                AppProfile(
                    businessName = profileDoc.getString("businessName") ?: "",
                    ownerName = profileDoc.getString("ownerName") ?: "",
                    phone = profileDoc.getString("phone") ?: "",
                    address = profileDoc.getString("address") ?: "",
                    gstNumber = profileDoc.getString("gstNumber") ?: "",
                    language = profileDoc.getString("language") ?: "en",
                    syncEnabled = true
                )
            } else null

            // Get customers
            val customersSnapshot = db.collection("users")
                .document(userId)
                .collection("customers")
                .get()
                .await()

            val customers = customersSnapshot.documents.mapNotNull { doc ->
                Customer(
```

```
                    id = doc.getLong("id") ?: 0L,
                    name = doc.getString("name") ?: return@mapNotNull null,
                    phone = doc.getString("phone") ?: "",
                    address = doc.getString("address") ?: "",
                    photoPath = doc.getString("photoPath"),
                    createdAt = doc.getLong("createdAt") ?:
System.currentTimeMillis()
                )
            }

            // Get transactions
            val transactionsSnapshot = db.collection("users")
                .document(userId)
                .collection("transactions")
                .get()
                .await()

            val transactions = transactionsSnapshot.documents.mapNotNull { doc ->
                TxnEntity(
                    id = doc.getLong("id") ?: 0L,
                    customerId = doc.getLong("customerId") ?: return@mapNotNull
null,
                    type = com.nammasanthe.ledger.data.entity.TxnType.valueOf(
                        doc.getString("type") ?: "CREDIT"
                    ),
                    amount = doc.getDouble("amount") ?: 0.0,
                    note = doc.getString("note"),
                    date = doc.getLong("date") ?: System.currentTimeMillis(),
                    createdAt = doc.getLong("createdAt") ?:
System.currentTimeMillis(),
                    syncPending = false,
                    photoPath = doc.getString("photoPath")
                )
            }

            _syncState.value = SyncState.Success

            Result.success(CloudData(profile, customers, transactions))
        } catch (e: Exception) {
            _syncState.value = SyncState.Error(e.message ?: "Download failed")
            Result.failure(e)
        }
    }

    private suspend fun syncProfile(userId: String, profile: AppProfile) {
        val profileData = hashMapOf(
            "businessName" to profile.businessName,
            "ownerName" to profile.ownerName,
            "phone" to profile.phone,
            "address" to profile.address,
            "gstNumber" to profile.gstNumber,
            "language" to profile.language,
            "lastUpdated" to System.currentTimeMillis()
        )

        db.collection("users")
            .document(userId)
            .collection("profile")
            .document("main")
            .set(profileData)
            .await()
    }

    private suspend fun syncCustomers(userId: String, customers: List<Customer>) {
        val batch = db.batch()
        val customersRef = db.collection("users")
            .document(userId)
            .collection("customers")

        customers.forEach { customer ->
            val data = hashMapOf(
                "id" to customer.id,
                "name" to customer.name,
                "phone" to customer.phone,
                "address" to customer.address,
                "photoPath" to (customer.photoPath ?: ""),
                "createdAt" to customer.createdAt,
                "lastUpdated" to System.currentTimeMillis()
            )
            batch.set(customersRef.document(customer.id.toString()), data,
SetOptions.merge())
```

```
        }

        batch.commit().await()
    }

    private suspend fun syncTransactions(userId: String, transactions:
List<TxnEntity>) {
        val batch = db.batch()
        val transactionsRef = db.collection("users")
            .document(userId)
            .collection("transactions")

        transactions.forEach { txn ->
            val data = hashMapOf(
                "id" to txn.id,
                "customerId" to txn.customerId,
                "type" to txn.type.name,
                "amount" to txn.amount,
                "note" to (txn.note ?: ""),
                "date" to txn.date,
                "createdAt" to txn.createdAt,
                "photoPath" to (txn.photoPath ?: ""),
                "lastUpdated" to System.currentTimeMillis()
            )
            batch.set(transactionsRef.document(txn.id.toString()), data,
SetOptions.merge())
        }

        batch.commit().await()
    }

    /**
     * Get sync status text.
     */
    fun getSyncStatusText(): String {
        return when (val state = _syncState.value) {
            is SyncState.Idle -> {
                val lastSync = _lastSyncTime.value
                if (lastSync != null) {
                    val dateFormat = SimpleDateFormat("dd MMM HH:mm",
Locale.getDefault())
                    "Last synced: ${dateFormat.format(Date(lastSync))}"
                } else "Not synced yet"
            }
            is SyncState.Syncing -> "Syncing..."
            is SyncState.Success -> "Sync completed"
            is SyncState.Error -> "Sync failed: ${state.message}"
        }
    }
}

/**
 * Sync states.
 */
sealed class SyncState {
    object Idle : SyncState()
    object Syncing : SyncState()
    object Success : SyncState()
    data class Error(val message: String) : SyncState()
}

/**
 * Data from cloud.
 */
data class CloudData(
    val profile: AppProfile?,
    val customers: List<Customer>,
    val transactions: List<TxnEntity>
)
```

5. UI Architecture (ViewModels)

MVVM implementation using Jetpack Compose StateFlow for reactive UI.

5.1 Core ViewModels

```
// LedgerViewModel.kt
package com.nammasanthe.ledger.viewmodel

import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import com.nammasanthe.ledger.data.entity.Customer
import com.nammasanthe.ledger.data.entity.TxnEntity
import com.nammasanthe.ledger.data.entity.TxnType
import com.nammasanthe.ledger.data.repo.LedgerRepository
import kotlinx.coroutines.flow.SharingStarted
import kotlinx.coroutines.flow.stateIn
import kotlinx.coroutines.launch

class LedgerViewModel(private val repo: LedgerRepository) : ViewModel() {
    val customers = repo.customers().stateIn(viewModelScope,
SharingStarted.Eagerly, emptyList())
    val balances = repo.balances().stateIn(viewModelScope, SharingStarted.Eagerly,
emptyList())
    val outstanding = repo.outstanding().stateIn(viewModelScope,
SharingStarted.Eagerly, 0.0)
    val recent = repo.recentTransactions(20).stateIn(viewModelScope,
SharingStarted.Eagerly, emptyList())

    fun customerTransactions(id: Long) = repo.customerTransactions(id)

    fun addCustomer(name: String, phone: String, address: String = "", photoPath:
String? = null) = viewModelScope.launch {
        repo.addCustomer(Customer(name = name, phone = phone, address = address,
photoPath = photoPath))
    }

    fun transactionsSince(since: Long) = repo.transactionsSince(since)

    fun addTransaction(customerId: Long, type: TxnType, amount: Double, note:
String? = null) =
        viewModelScope.launch {
            repo.addTransaction(
                TxnEntity(customerId = customerId, type = type, amount = amount,
note = note)
            )
        }

    suspend fun addCredit(cId: Long, amount: Double, note: String?, photoPath:
String? = null): Long {
        val txn = TxnEntity(customerId = cId, type = TxnType.CREDIT, amount =
amount, note = note, photoPath = photoPath)
        return repo.addTransaction(txn)
    }

    suspend fun addPayment(cId: Long, amount: Double, note: String?, photoPath:
String? = null): Long {
        val txn = TxnEntity(customerId = cId, type = TxnType.PAYMENT, amount =
amount, note = note, photoPath = photoPath)
        return repo.addTransaction(txn)
    }

    suspend fun getTransactionById(txnId: Long): TxnEntity? {
        return repo.getTransactionById(txnId)
    }

    suspend fun updateTransactionPhoto(transactionId: Long, photoPath: String) {
        repo.updateTransactionPhoto(transactionId, photoPath)
    }

    /** Add a transaction and return the new row ID (for QR generation). */
    suspend fun addTransactionGetId(
        customerId : Long,
        type       : TxnType,
        amount     : Double,
        note       : String? = null
    ): Long = repo.addTransaction(
        TxnEntity(customerId = customerId, type = type, amount = amount, note =
note)
    )

    suspend fun getCustomer(id: Long) = repo.getCustomer(id)

    suspend fun getAllTransactions(): List<TxnEntity> {
        return repo.getAllTransactions()
    }
```

```
    fun deleteCustomer(c: Customer) = viewModelScope.launch {
repo.deleteCustomer(c) }
}
```

```
// OcrViewModel.kt
package com.nammasanthe.ledger.viewmodel

import android.content.Context
import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import com.nammasanthe.ledger.NammaSantheApp
import com.nammasanthe.ledger.ai.EnhanceEngine
import com.nammasanthe.ledger.camera.decodeAndOrient
import com.nammasanthe.ledger.data.entity.ScanData
import com.nammasanthe.ledger.ocr.OcrLanguage
import com.nammasanthe.ledger.ocr.OcrPipeline
import com.nammasanthe.ledger.ocr.OcrResult
import com.nammasanthe.ledger.ocr.BillItem
import com.nammasanthe.ledger.ocr.BillParser
import com.nammasanthe.ledger.translation.KannadaOfflineTranslator
import com.nammasanthe.ledger.translation.TranslationManager
import com.nammasanthe.ledger.util.ImageStore
import com.nammasanthe.ledger.gemini.GeminiService
import com.nammasanthe.ledger.gemini.GeminiSettingsStore
import com.nammasanthe.ledger.gemini.GeminiOcrResult
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.launch
import kotlinx.coroutines.withContext
import java.io.File

data class OcrUiState(
    val processing: Boolean = false,
    val original: String = "",
    val translated: String = "",
    val language: OcrLanguage = OcrLanguage.UNKNOWN,
    val confidence: Float = 0f,
    val source: String = "",
    val amount: Double? = null,
    val dateMillis: Long? = null,
    val warning: String? = null,
    val error: String? = null,
    val imagePath: String? = null,
    val parsedItems: List<com.nammasanthe.ledger.ocr.BillItem> = emptyList(),
    val usingGemini: Boolean = false
)

class OcrViewModel(private val app: NammaSantheApp) : ViewModel() {

    private val pipeline = OcrPipeline(app)
    private val translator = TranslationManager()
    private val knTranslator = KannadaOfflineTranslator()
    private val enhancer = EnhanceEngine()
    private val billParser = BillParser(knTranslator)
    private val geminiService = GeminiService()
    private val geminiSettings = GeminiSettingsStore(app)

    private val _state = MutableStateFlow(OcrUiState())
    val state = _state.asStateFlow()

    fun setPreferredLanguage(language: OcrLanguage) {
        pipeline.setPreferredLanguage(language)
    }

    fun processCapturedFile(file: File) {
        viewModelScope.launch {
            _state.value = _state.value.copy(processing = true, error = null,
warning = null, usingGemini = false)
            try {
                val savedPath = withContext(Dispatchers.IO) {
                    ImageStore.saveCompressed(app, file)
                }
                val bitmap = withContext(Dispatchers.IO) { decodeAndOrient(file) }

                // Check if Gemini is enabled and available
                val useGemini = geminiSettings.isGeminiAvailable()

                val ocr: OcrResult
                val geminiResult: GeminiOcrResult?

                if (useGemini) {
                    _state.value = _state.value.copy(usingGemini = true)
                    val apiKey = geminiSettings.getApiKey()
```

```
                    geminiResult = geminiService.performOcr(bitmap, apiKey)

                    if (geminiResult.success) {
                        // Convert Gemini result to OcrResult format
                        ocr = OcrResult(
                            rawText = geminiResult.rawText,
                            language = OcrLanguage.KANNADA,
                            confidence = 0.95f,
                            source = "gemini-api",
                            amount = extractAmountFromItems(geminiResult.items),
                            dateMillis = null,
                            warning = null
                        )

                        // Parse Gemini items directly
                        val geminiItems = geminiResult.items.map { item ->
                            com.nammasanthe.ledger.ocr.BillItem(
                                item = item.name,
                                originalLine = "${item.name} ${item.quantity}
${item.amount}",
                                quantity = item.quantity,
                                amount = item.amount,
                                confidence = 0.9f
                            )
                        }

                        _state.value = OcrUiState(
                            processing = false,
                            original = geminiResult.rawText,
                            translated = geminiResult.translatedText,
                            language = OcrLanguage.KANNADA,
                            confidence = 0.95f,
                            source = "gemini-api",
                            amount = extractAmountFromItems(geminiResult.items),
                            warning = null,
                            imagePath = savedPath,
                            parsedItems = geminiItems,
                            usingGemini = true
                        )
                        return@launch
                    } else {
                        // Fall back to on-device OCR if Gemini fails
                        geminiResult.error?.let {
                            _state.value = _state.value.copy(
                                warning = "Gemini API failed: $it. Falling back to
on-device OCR."
                            )
                        }
                        ocr = pipeline.run(bitmap)
                    }
                } else {
                    geminiResult = null
                    ocr = pipeline.run(bitmap)
                }
                val translation = when (ocr.language) {
                    OcrLanguage.ENGLISH -> ocr.rawText
                    OcrLanguage.HINDI -> pipeline.translateIfNeeded(ocr,
translator)
                    OcrLanguage.KANNADA -> knTranslator.translate(ocr.rawText)
                    else -> ""
                }
                val warnings = mutableListOf<String>()
                ocr.warning?.let { warnings.add(it) }
                when (ocr.language) {
                    OcrLanguage.HINDI -> {
                        if (ocr.rawText.isNotBlank() &&
translation.isNullOrBlank()) {
                            val modelReady = translator.isModelDownloaded("hi") &&
translator.isModelDownloaded("en")
                            val msg = if (modelReady) {
                                "Translation failed. Tap Translate Again."
                            } else {
                                "Hindi translation model not downloaded. Connect
once to download for offline use."
                            }
                            warnings.add(msg)
                        }
                    }
                    OcrLanguage.KANNADA -> {
                        if (!knTranslator.hasChanges(ocr.rawText, translation ?:
"")) {
```

```
                            warnings.add("Kannada translation is limited. Edit
manually if needed.")
                        }
                    }
                    else -> Unit
                }
                val warningText = warnings.joinToString("\n").ifBlank { null }
                val translatedForParsing = translation ?: ocr.rawText
                val parsedItems = withContext(Dispatchers.Default) {
                    billParser.parseTranslated(translatedForParsing)
                }
                _state.value = OcrUiState(
                    processing   = false,
                    original     = ocr.rawText,
                    translated   = translation ?: "",
                    language     = ocr.language,
                    confidence   = ocr.confidence,
                    source       = ocr.source,
                    amount       = ocr.amount,
                    dateMillis   = ocr.dateMillis,
                    warning      = warningText,
                    imagePath    = savedPath,
                    parsedItems  = parsedItems
                )
            } catch (t: Throwable) {
                _state.value = _state.value.copy(processing = false, error =
t.message, warning = null)
            }
        }
    }

    fun retranslate() {
        viewModelScope.launch {
            val s = _state.value
            if (s.original.isBlank()) return@launch
            _state.value = s.copy(processing = true, error = null)
            try {
                when (s.language) {
                    OcrLanguage.ENGLISH -> {
                        _state.value = s.copy(processing = false, translated =
s.original, warning = null)
                    }
                    OcrLanguage.HINDI -> {
                        val translated = translator.translate(s.original, "hi",
"en")
                        _state.value = s.copy(processing = false, translated =
translated, warning = null)
                    }
                    OcrLanguage.KANNADA -> {
                        val translated = knTranslator.translate(s.original)
                        val warn = if (knTranslator.hasChanges(s.original,
translated)) null
                        else "Kannada translation is limited. Edit manually if
needed."
                        _state.value = s.copy(processing = false, translated =
translated, warning = warn)
                    }
                    else -> {
                        _state.value = s.copy(
                            processing = false,
                            warning = "Translation is not available for this
language."
                        )
                    }
                }
            } catch (t: Throwable) {
                val msg = t.message?.takeIf { it.isNotBlank() }
                    ?: "Translation failed. Connect once to download the model."
                _state.value = _state.value.copy(processing = false, warning =
msg)
            }
        }
    }

    /**
     * Re-parse the current translated (or original) text into structured
[BillItem]s.
     * Called automatically after [processCapturedFile] and also available as a
manual
     * "Re-Parse" action in the UI.
     */
```

```
    fun parseBill() {
        viewModelScope.launch {
            val s = _state.value
            val text = s.translated.ifBlank { s.original }
            if (text.isBlank()) return@launch
            _state.value = s.copy(processing = true)
            val items = withContext(Dispatchers.Default) {
                billParser.parseTranslated(text)
            }
            _state.value = _state.value.copy(processing = false, parsedItems =
items)
        }
    }

    fun enhance() {
        viewModelScope.launch {
            val s = _state.value
            if (s.original.isBlank()) return@launch
            _state.value = s.copy(processing = true)
            val cleaned = enhancer.cleanup(s.original)
            val summary = enhancer.summarize(cleaned)
            _state.value = _state.value.copy(processing = false, original =
cleaned, translated = summary)
        }
    }

    fun updateOriginal(text: String) {
        _state.value = _state.value.copy(original = text)
    }

    fun updateTranslated(text: String) {
        _state.value = _state.value.copy(translated = text)
    }

    fun saveScan(customerId: Long? = null) {
        val s = _state.value
        if (s.imagePath == null || s.original.isBlank()) return
        viewModelScope.launch {
            app.repository.saveScan(
                ScanData(
                    imagePath = s.imagePath,
                    originalText = s.original,
                    translatedText = s.translated.ifBlank { null },
                    detectedLanguage = s.language.name,
                    extractedAmount = s.amount,
                    extractedDate = s.dateMillis,
                    customerId = customerId
                )
            )
        }
    }

    override fun onCleared() {
        pipeline.close()
        translator.close()
        // GeminiService doesn't require explicit cleanup
    }

    @Suppress("unused")
    fun context(): Context = app

    /**
     * Extract total amount from Gemini bill items.
     */
    private fun extractAmountFromItems(items:
List<com.nammasanthe.ledger.gemini.BillItem>): Double? {
        if (items.isEmpty()) return null
        return items.sumOf { it.amount.toDouble() }
    }
}
```